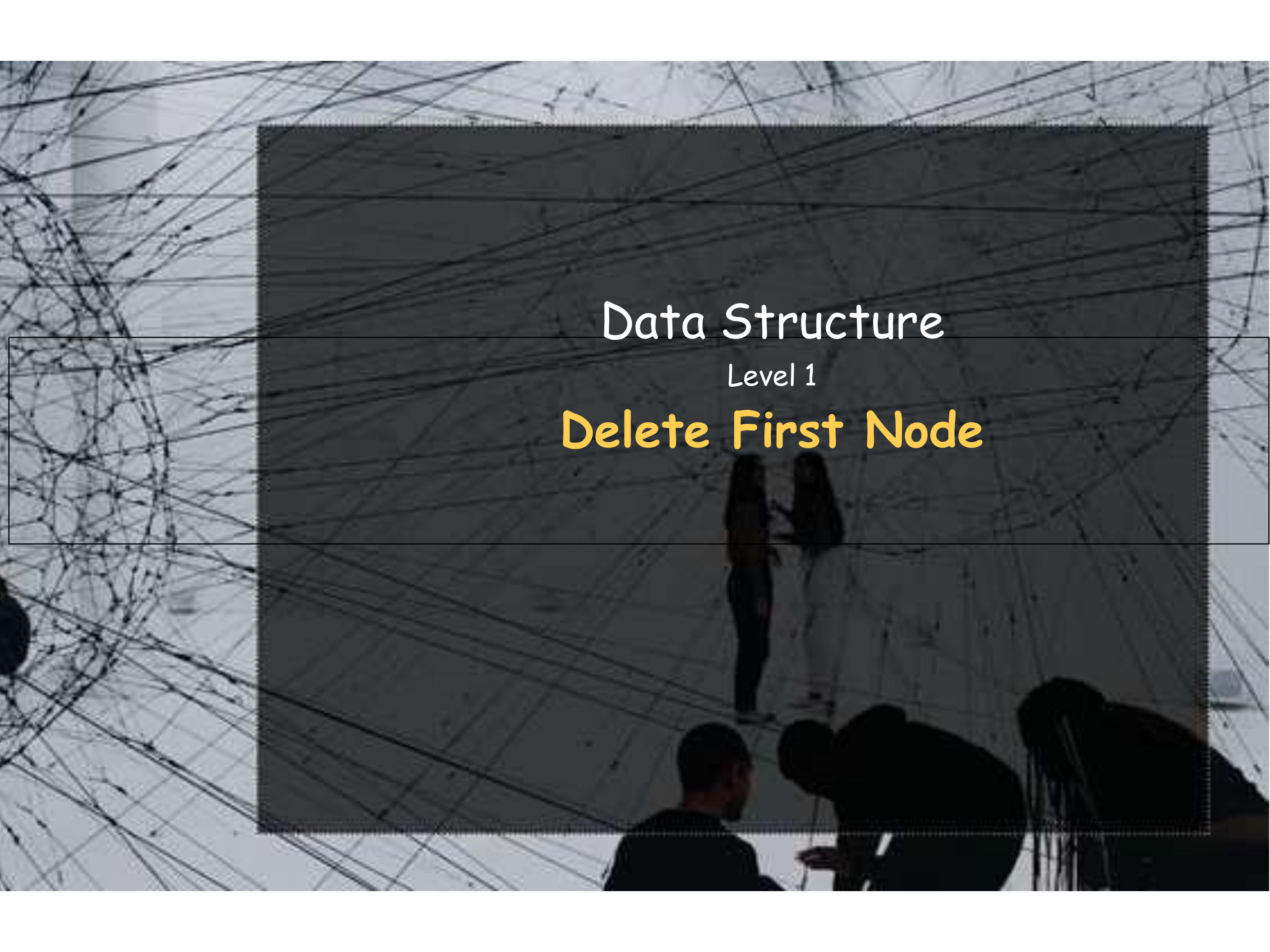


Data Structure

Level 1

Singly Linked List Data Structures Operations

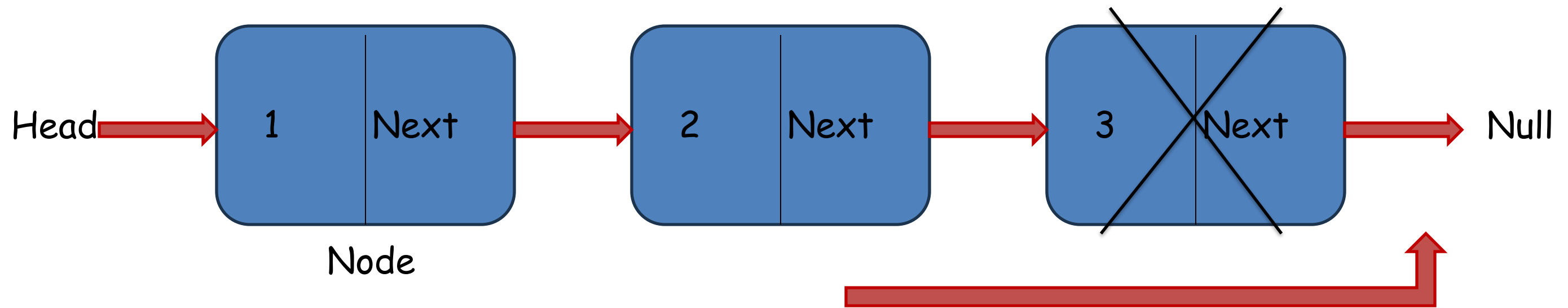


Data Structure

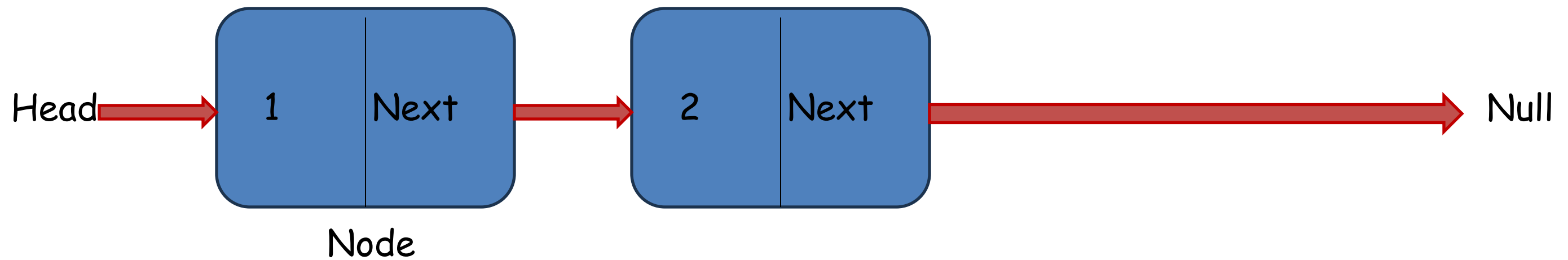
Level 1

Delete First Node

Linked List Data Structure



Linked List Data Structure



Definition

DeleteLastNode() Function:

Removes the last node from the linked list and frees its memory.

■ Special Cases:

- If list is empty → do nothing
- If list has one node → delete it and set head to NULL
- Otherwise → traverse to last node and remove it



Explanation

- ◆ Two pointers are used:
 - Current → moves through the list
 - Prev → stores the previous node
- ◆ Traverse until `Current->next == NULL` (last node).
- ◆ Set `Prev->next = NULL`.
- ◆ Delete the last node using delete.

Procedures (Delete Last Node)

- 1 Check if list is empty
- 2 If only one node → delete it and set head = NULL
- 3 Initialize Current = head and Prev = head
- 4 Traverse until reaching last node
- 5 Move Prev and Current forward
- 6 Stop when Current->next == NULL
- 7 Set Prev->next = NULL
- 8 Delete Current
- 9 Print updated list



Example

Initial list:

1 → 2 → 3 → 4 → 5 → 6 → NULL

After calling DeleteLastNode(head)

➡ Resulting list:

1 → 2 → 3 → 4 → 5 → NULL



Note

Deleting the last node requires traversal because singly linked lists do not store a pointer to the previous node automatically.



Comparison

- Delete First Node
 - ✓ Time Complexity: $O(1)$
 - ✓ No traversal needed
 - ✓ Simple pointer update

- Delete Last Node
 - ✓ Time Complexity: $O(n)$
 - ✓ Requires traversal
 - ✓ Uses two pointers

Warning

If you do not handle the single-node case, the program may crash.
Always update Prev->next before deleting the last node.

✨ Summary

- ✓ DeleteLastNode removes the final node
- ✓ Requires traversal ($O(n)$)
- ✓ Handles empty and single-node cases
- ✓ Frees memory properly using delete
- ✓ Demonstrates forward-only nature of singly linked lists

Thank you 😊